Greedy Algorithms

(continued)

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

Greedy Algorithm Example

Huffman Codes

TEXT ENCODING

- **Our next goal**
  - ✓ is to develop a code that represents a given text as compactly as possible.

- **A standard encoding is ASCII**
  - ✓ which represents every character using 7 bits (early in its development)

  - ✓ A phrase "An English sentence"
    - 1000001 (A) 1101110 (n) 0100000 ( ) 1000101 (E)
      1101110 (n) 1100111 (g) 1101100 (l) 1101001 (i)
      1110011 (s) 1101000 (h) 0100000 ( ) 1110011 (s)
      1100101 (e) 1101110 (n) 1110100 (t) 1100101 (e)
      1101110 (n) 1100011 (c) 1100101 (e)
    - This requires 133 bits ≈ **17 bytes**

TEXT ENCODING

- **We can encode 12 characters in 4 bits:**
  - ‹space› = 0000 A = 0001 E = 0010 c = 0011 e = 0100 g = 0101
    h = 0110 i = 0111 l = 1000 n = 1001 s = 1010 t = 1011
  - Then we encode the phrase "An English sentence" as
    - 0001 (A) 1001 (n) 0000 ( ) 0010 (E) 1001 (n) 0101 (g) 1000 (l)
      0111 (i) 1010 (s) 0110 (h) 0000 ( ) 1010 (s) 0100 (e) 1001 (n)
      1011 (t) 0100 (e) 1001 (n) 0011 (c) 0100 (e)
    - This requires 76 bits ≈ **10 bytes**

TEXT ENCODING

- **An even better code is given by the following encoding:**
  - ‹space› = 000 A = 0010 E = 0011 s = 010 c = 0110
    g = 0111 h = 1000 i = 1001 l = 1010 t = 1011 e = 110 n = 111
  - Then we encode the phrase "An English sentence" as
    - 0010 (A) 111 (n) 000 ( ) 0011 (E) 111 (n) 0111 (g) 1010 (l) 1001 (i)
      010 (s) 1000 (h) 000 ( ) 010 (s) 110 (e) 111 (n) 1011 (t) 110 (e)
      111 (n) 0110 (c) 110 (e)
    - This requires 65 bits ≈ **9 bytes**

Prefix Code

❖ Prefix code

- ✓ The optimal data compression achievable by a prefix code.
- ✓ **No codeword** is a **prefix** of some other codewords.
- ✓ Easy to decode

	a	b	c	d	e	f
Variable – length code	0	101	100	111	1101	1100

a codeword

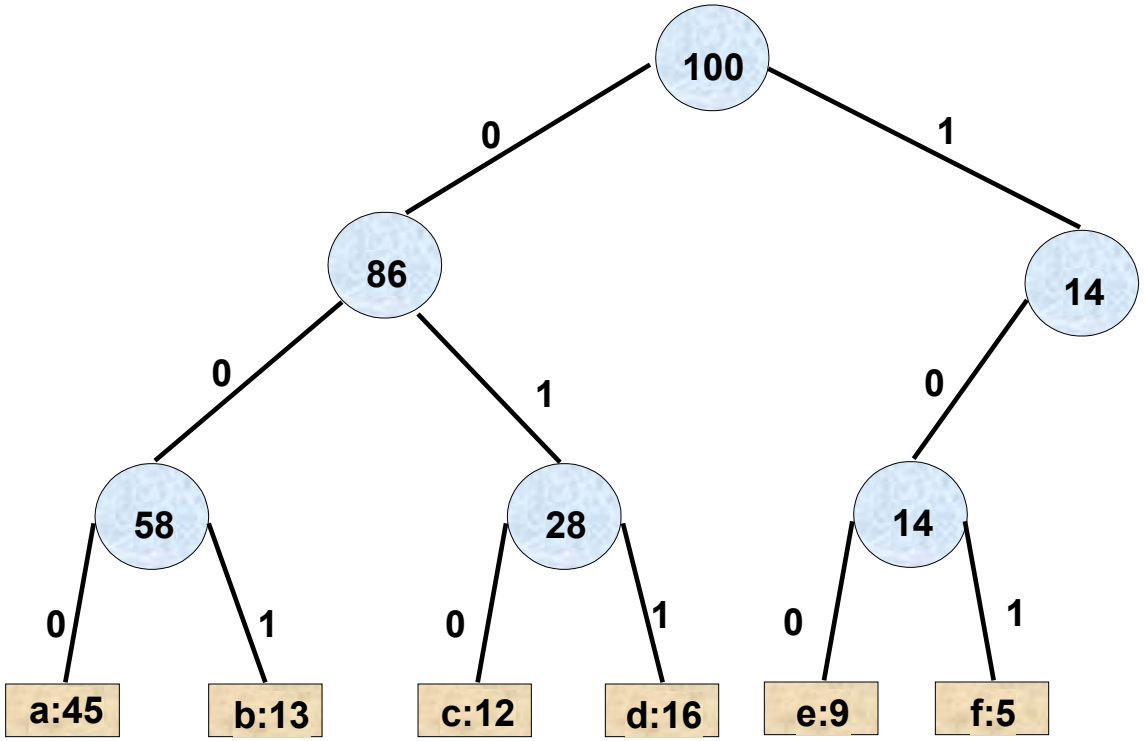
001011101→aabe

WHY PREFIX CODES?

- **What if we have a code that is not a prefix code?**
  - ✓ a = 01 m = 10 n = 111 o = 0 r = 11 s = 1 t = 0011
- Now you send a fan-letter to your favorite movie star.
  - ✓ One of the sentences is **"You are a star."**
  - ✓ You encode "star" as "1 0011 01 11".
- Your idol receives the letter and decodes the text using your coding table:
  - ✓ 100110111 = 10 0 11 0 111 = "moron"
  - ✓ Oops, you have just insulted your idol.
- Non-prefix codes are ambiguous.

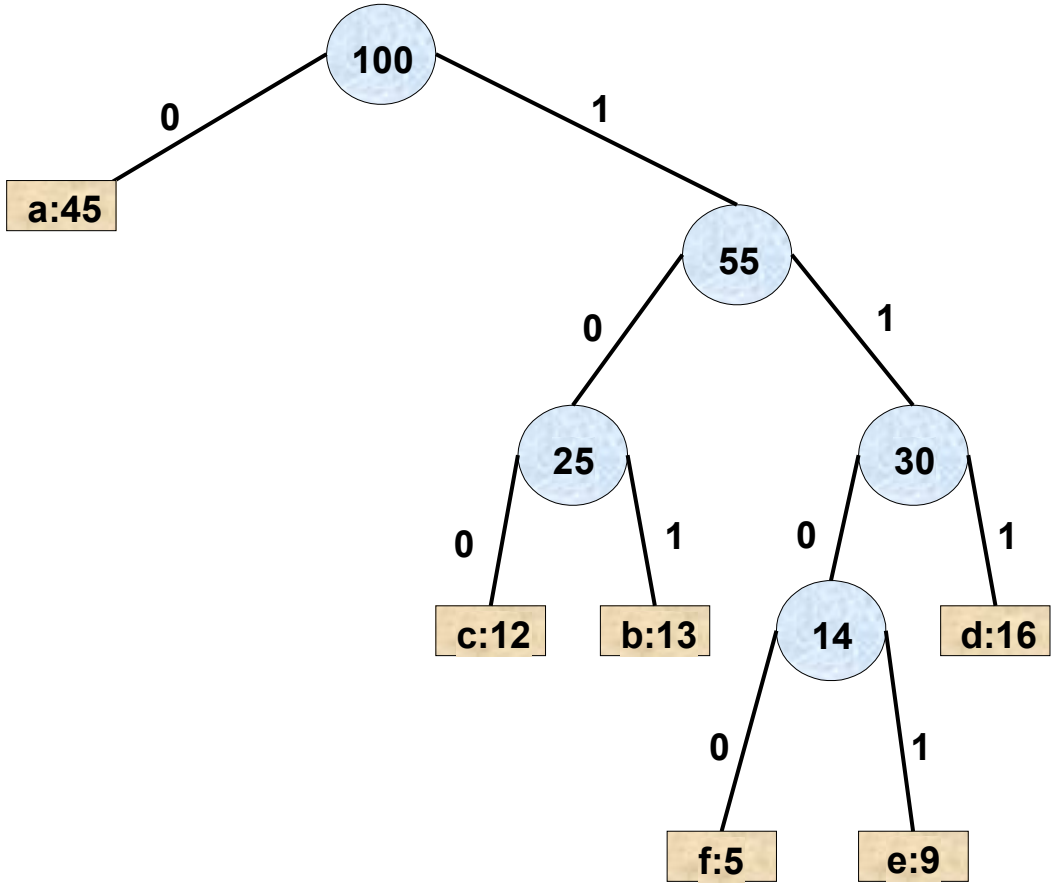
DECODING/ENCODING TREE : FIXED-LENGTH CODE

	a	b	c	d	e	f
Frequency	45	13	12	16	9	5
Fixed-length code	000	001	010	011	100	101



DECODING/ENCODING TREE : VARIABLE-LENGTH CODE

	a	b	c	d	e	f
Frequency	45	13	12	16	9	5
Variable-length code	0	101	100	111	1101	1100



COST OF ENCODING A FILE

- T: tree corresponding to a coding scheme (prefix code)
- c: a character in the alphabet C.
- $f(c)$: the frequency of c in the file.
- $d_T(c)$: the depth of c's leaf in the tree. (code length)
- $B(T)$: the number of bits required to encode a file.

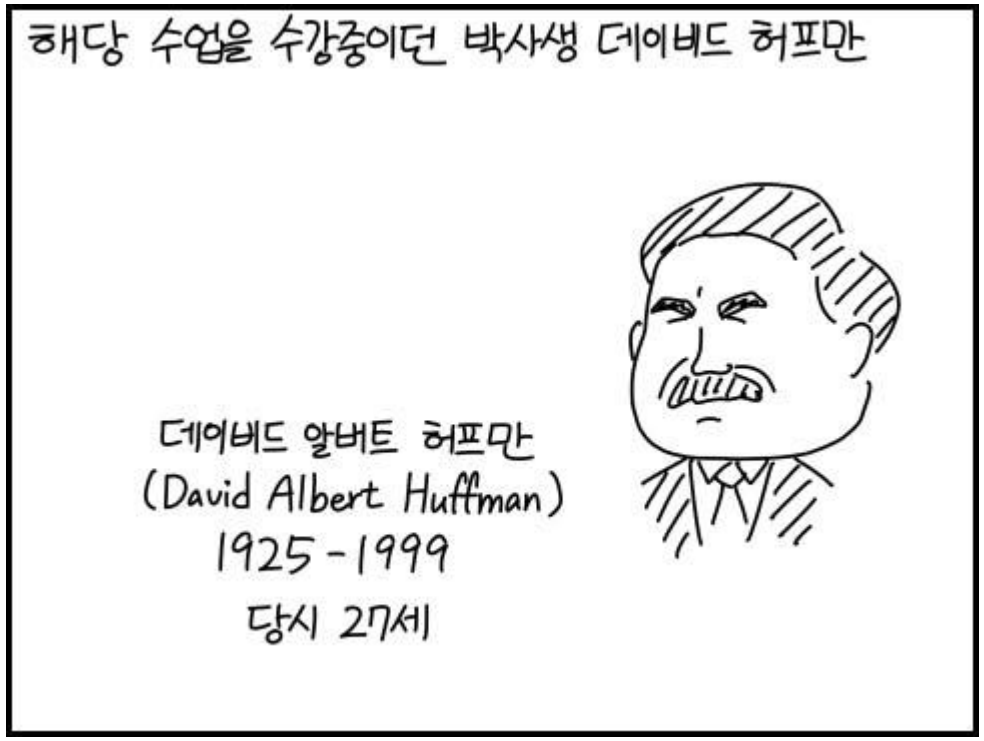
$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

❖ Invented by Huffman

"기말고사를 피하는 방법> - 야밤의 공대생 만화
(A fun story in Korean about how the Huffman code method was developed.)
https://www.facebook.com/photo?fbid=850293368490853&set=pcb.850294048490785

HUFFMAN CODES

- Very effective technique for compressing data
- An optimal prefix code
- Greedy algorithm!



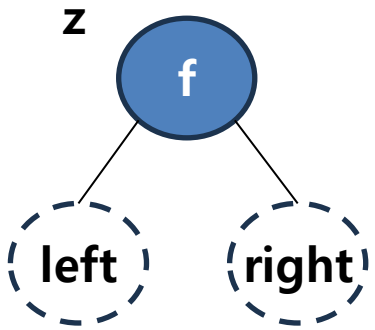
HUFFMAN'S ALGORITHM

❖ **Huffman-Code(C)**

```
n ← |C|
Q ← C
for i = 1 … n - 1
  do allocate a new node z
    left[z] ← Extract-Min(Q)
    right[z] ← Extract-Min(Q)
    f[z] ← f[left[z]] + f[right[z]]
    Insert(Q, z)
return Extract-Min(Q)
```

a node **z**

- frequency, f
- left child node
- right child node



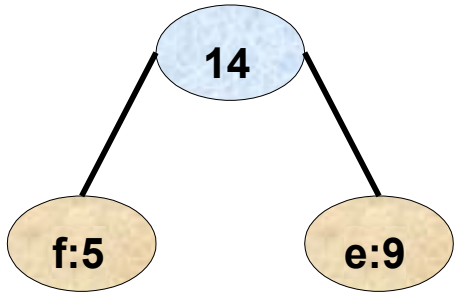
HUFFMAN TREE: EXAMPLE

Step 1:

	a	b	c	d	e	f
Frequence	45	13	12	16	9	5

Step 2:

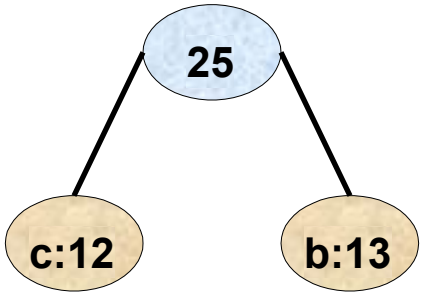
	a	b	c	d	
Frequence	45	13	12	16	14



	a	d		
Frequence	45	16	14	25

❖ Huffman-Code(C)

```
n ← |C|
Q ← C
for i = 1 … n – 1
  do allocate a new node z
    left[z] ← Extract-Min(Q)
    right[z] ← Extract-Min(Q)
    f[z] ← f[left[z]] + f[right[z]]
    Insert(Q, z)
return Extract-Min(Q)
```

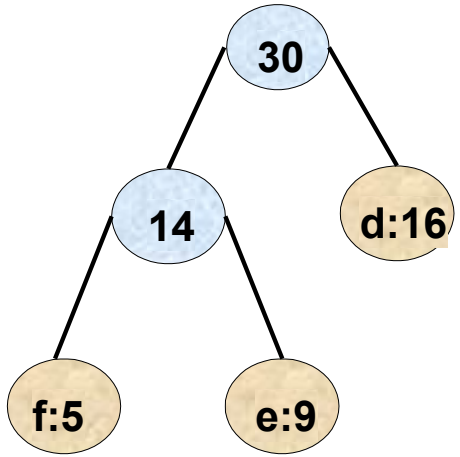


HUFFMAN TREE: EXAMPLE

Step 3:

	a	d		
Frequence	45	16	14	25

	a		
Frequence	45	25	30



❖ **Huffman-Code(C)**

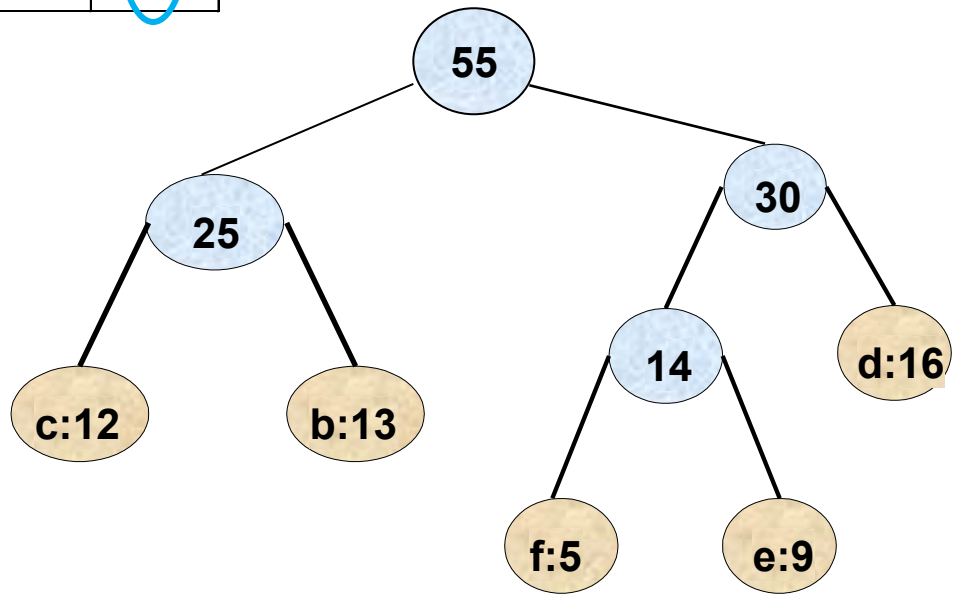
```
n ← |C|
Q ← C
for i = 1 … n – 1
  do allocate a new node z
    left[z] ← Extract-Min(Q)
    right[z] ← Extract-Min(Q)
    f[z] ← f[left[z]] + f[right[z]]
    Insert(Q, z)
return Extract-Min(Q)
```

HUFFMAN TREE: EXAMPLE

Step 4:

	a		
Frequence	45	25	30

	a	
Frequence	45	55



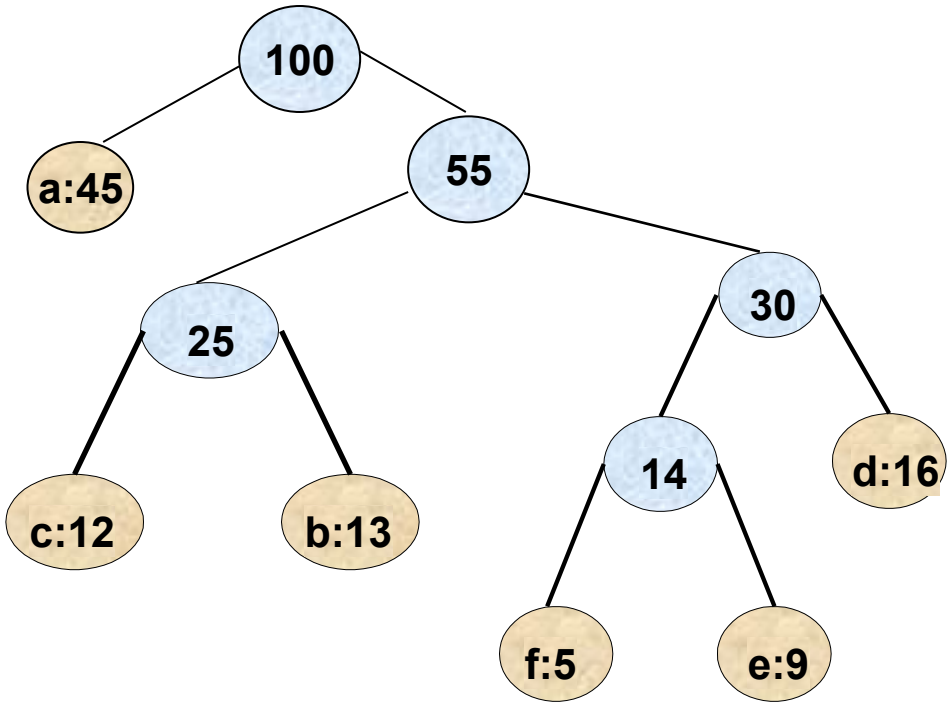
❖ **Huffman-Code(C)**

```
n ← |C|
Q ← C
for i = 1 … n – 1
  do allocate a new node z
    left[z] ← Extract-Min(Q)
    right[z] ← Extract-Min(Q)
    f[z] ← f[left[z]] + f[right[z]]
    Insert(Q, z)
return Extract-Min(Q)
```

HUFFMAN TREE: EXAMPLE

Step 5:

	a	
Frequence	45	55



❖ Huffman-Code(C)

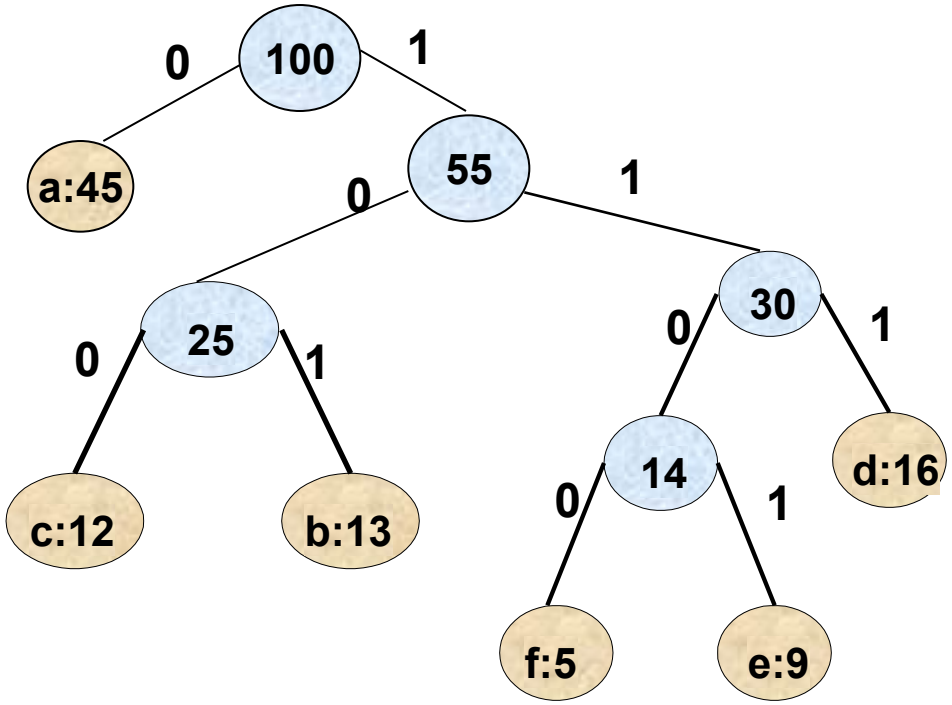
```
n ← |C|
Q ← C
for i = 1 … n – 1
  do allocate a new node z
    left[z] ← Extract-Min(Q)
    right[z] ← Extract-Min(Q)
    f[z] ← f[left[z]] + f[right[z]]
    Insert(Q, z)
return Extract-Min(Q)
```

HUFFMAN TREE: EXAMPLE

Coding scheme

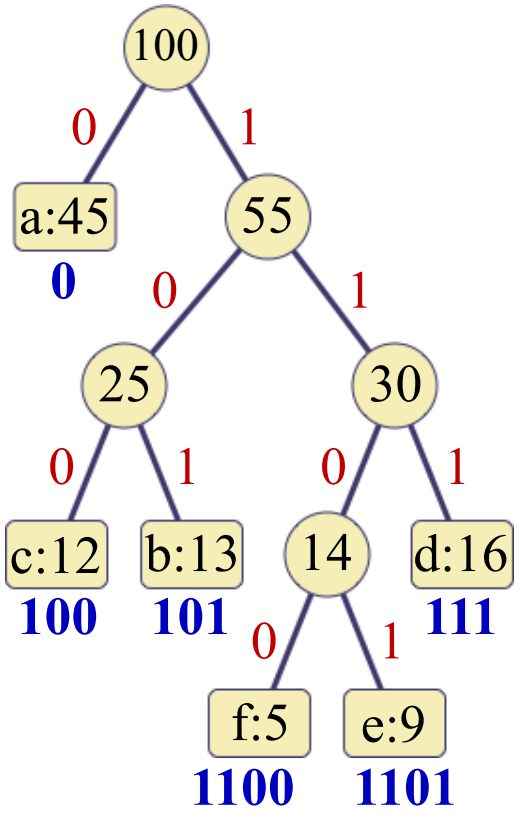
0 for the left branch

1 for the right branch



❖ **Huffman-Code(C)**

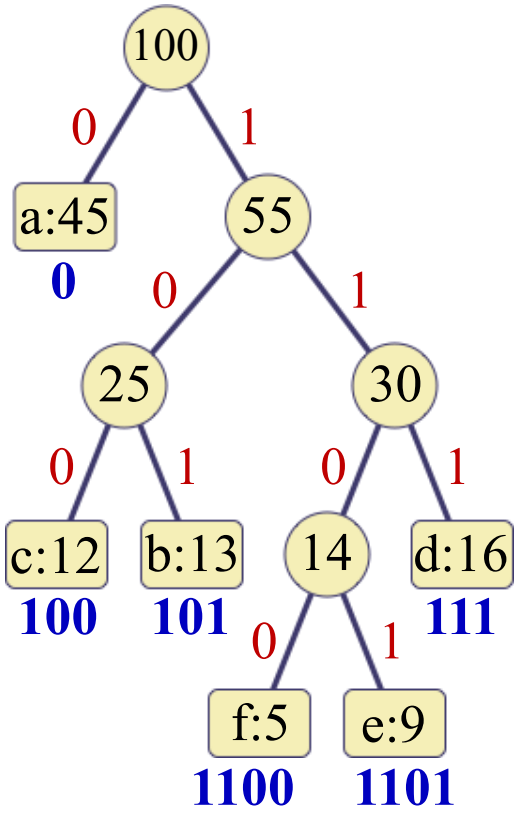
```
n ← |C|
Q ← C
for i = 1 … n - 1
  do allocate a new node z
    left[z] ← Extract-Min(Q)
    right[z] ← Extract-Min(Q)
    f[z] ← f[left[z]] + f[right[z]]
    Insert(Q, z)
return Extract-Min(Q)
```



Running time: $O(n \cdot lg(n))$

assuming that finding the minimum requires $O(\lg n)$

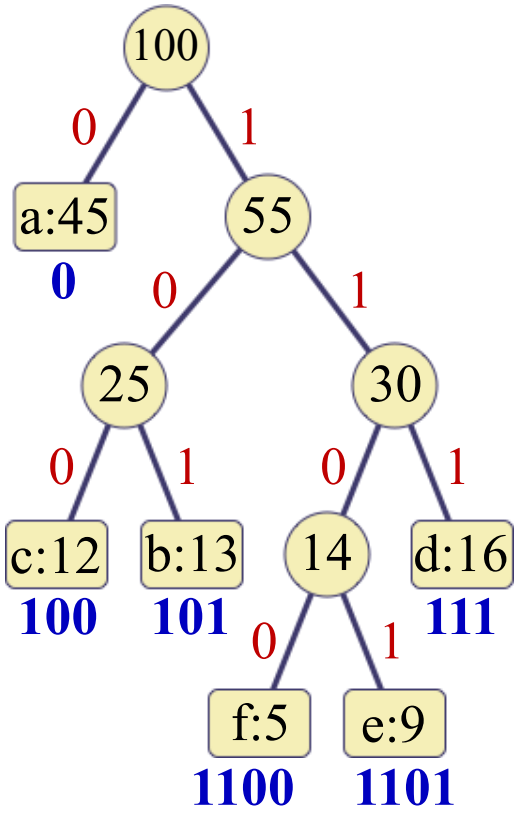
- **Encoding**
  - ✓ Convert the tree into a lookup table.



a	b	c	d	E	f
0	101	100	111	1101	1100

❖ Decoding

- ✓ Traverse the tree.
- ✓ Since we use the prefix code,
  we follow the tree from the root to each leaf.
- ✓ For each bit read,
  go to the left child at that level if it is 0
  or go to the right child at that level if it is 1.
- ✓ When a leaf node is reached,
  the corresponding character is identified..
- ✓ Return to the root to decode the next character.



HW 4

HUFFMAN'S ALGORITHM

- **Draw the optimal Huffman Tree for the given string below.**

"Why not change the world?"

(Note: Treat uppercase/lowercase letters, spaces, and the question mark as distinct characters.)

- **Generate the corresponding prefix code table based on your tree.**
- **Encode the entire phrase into a binary bitstream using the table.**

- **Requirements**
  - ✓ You must show the entire process of deriving your results.
  - ✓ Assign binary codes (0 for left, 1 for right) to each character based on the tree structure.
  - ✓ Due: 4/14(Tue) 2:30pm

Thank you!